



Red Hat Enterprise Linux 8

RHCSA Topics Commands

Eng. Muhammad Adel

redhat.
CERTIFIED
SYSTEM
ADMINISTRATOR



Managing users and Groups

Users:

```
[root@node1 ~]# ls -l
```

```
[root@node1 ~]# useradd --help
```

```
[root@node1 ~]# useradd ali
```

```
[root@node1 ~]# passwd ali
```

```
[root@node1 ~]# useradd -G admin -u 1005 -s /usr/sbin/nologin ali
```

```
[root@node1 ~]# useradd -c "Muhammad Adel" -e 2020-10-16 -s /sbin/nologin adel
```

To verify:

```
[root@node1 ~]# id
```

```
[root@node1 ~]# id ali
```

```
[root@node1 ~]# id -u ali      (UID for ali)
```

```
[root@node1 ~]# tail -n 1 /etc/passwd
```

```
Login-name:password:UID:GID:GECOS:/home/dir:shell
```

HINT:

GECOS field is arbitrary text, which usually includes the user's real name.

```
[root@node1 ~]# tail -n 1 /etc/shadow
```

```
name:password:1astchange:minage:maxage:warning:inactive:expire:blank
```

HINT:

!! indicates that the user has no password

Groups:

- Primary group is the user effective group

```
[root@node1 ~]# groupadd sales
```

```
[root@node1 ~]# groupadd -g 1005 admin
```

To verify:

```
[root@node1 ~]# id
```

```
[root@node1 ~]# id ali
```

```
[root@node1 ~]# grep sales /etc/group
```

```
groupname:password:GID:<members of this group>
```

```
[root@node1 ~]# groupmod -g 2000 admin
```

```
[root@node1 ~]# groupadd old
```

```
[root@node1 ~]# groupmod -n new old    (rename a group)
```

Switching users with su:

```
[root@node1 ~]# su ali
```

```
[ali@node1 root]$ exit
```

```
exit
```

```
[root@node1 ~]# su - ali
```

```
[ali@node1 ~]$
```

```
[ali@node1 ~]$ su
```

```
[ali@node1 ~]$ su -
```

Running commands as root with sudo:

```
[root@node1 ~]# vim /etc/sudoers
```

```
ali  ALL=(ALL)    ALL
```

```
%sales  ALL=(ALL)    ALL
```

```
Ansone      ALL=(ALL)    NOPASSWD: ALL
```

```
%wheel      ALL=ALL      ALL
```

```
[ali@node1 ~]$ sudo passwd ahmed
```

```
[ali@node1 ~]$ sudo passwd -l ahmed
```

To verify:

```
[root@node1 ~]# tail -f /var/log/secure
```

Modify users:

```
[root@node1 ~]# usermod -L ali          (lock the user)
```

```
[root@node1 ~]# usermod -U ali          (unlock the user)
```

```
[root@node1 ~]# usermod -G sales ali     (overwrite secondary group)
```

```
[root@node1 ~]# usermod -aG admin ali          (append to secondary group)
```

or:

```
[root@node1 ~]# vim /etc/group
```

To verify:

```
[root@node1 ~]# id ali
```

Delete users:

```
[root@node1 ~]# userdel ali
```

```
[root@node1 ~]# userdel -r test    (removes home directory)
```

Delete groups:

```
[root@node1 ~]# groupdel admin
```

UID ranges:

UID 0 is always assigned to the superuser account, root.

UID 1-200 is a range of "system users" assigned statically to system processes by Red Hat.

UID 201-999 is a range of "system users" used by system processes that do not own files on the file system.

UID 1000+ is the range available for assignment to regular users.

To change the default:

```
[root@node1 ~]# vim /etc/login.defs
```

Password aging:

```
[root@node1 ~]# chage -l ali    (list info about the user)
```

```
[root@node1 ~]# chage -E 2017-1-1 ali    (expire the user on the specified address)
```

```
[root@node1 ~]# chage -m 1 ali    (set minimum number of days before password change)
```

```
[root@node1 ~]# chage -M 120 ali    (set maximum number of days before password change)
```

```
[root@node1 ~]# passwd -x 90 ali    (the password will expire after 90 days)
```

Access Permissions

- Only the root and the owner can change the permissions.

```
[root@node1 ~]# ls -l file    OR  [root@node1 ~]# ll file
```

```
[root@node1 ~]# ls -ld /home
```

Changing file/directory permissions:

Symbolic method:

- Who is u, g, o, a (for user, group, other, all)
- What is +, -, = (for add, remove, set exactly)
- Which is r, w, x (for read, write, executable)

```
[root@node1 ~]# chmod g+w file1
```

```
[root@node1 ~]# chmod o+w file1
```

```
[root@node1 ~]# chmod u-w file1
```

```
[root@node1 ~]# chmod u+w,g+wx,o+r file1
```

```
[root@node1 ~]# chmod go-rw file1
```

```
[root@node1 ~]# chmod u=rw,g=r,o=r file1    (resets all old permissions)
```

```
[root@node1 ~]# chmod a+x file1    or    chmod ugo+x file1
```

```
[root@node1 ~]# chmod a=rw file1    or    chmod ugo=rw file1
```

```
[root@node1 ~]# chmod u= file1    (removes all permissions from owner)
```

```
[root@node1 ~]# chmod +rw file1    or    chmod u+rw file1
```

```
[root@node1 ~]# chmod =rw file1    or    chmod u=rw file1
```

```
[root@node1 ~]# chmod -R g+rw dir1
```

Numeric method:

r=4, w=2, x=1

```
[root@node1 ~]# chmod 754 file1    (rwx,r-x,r--)
```

```
[root@node1 ~]# chmod 400 file1    (r--,---,---)
```

```
[root@node1 ~]# chmod -R 755 dir1
```

Changing file/directory user or group ownership:

- Only root can change the ownership of a file.

- Root or the file's owner can change group ownership.

```
[root@node1 ~]# chown ali file1
```

```
[root@node1 ~]# chown ali dir1
```

```
[root@node1 ~]# chown -R ali dir1
```

```
[root@node1 ~]# chown :sales file1      (change the group ownership)
```

```
[root@node1 ~]# chgrp sales file1      (change the group ownership)
```

```
[root@node1 ~]# chown ali:sales file1   (change the owner and group)
```

```
[root@node1 ~]# chown -R ali:sales dir1
```

Special permissions:

- The setuid (or setgid) permission on an executable file means that the command will run as the user (or group) of the file, not as the user that ran the command.

```
[root@node1 ~]# ls -l /usr/bin/passwd
```

-The sticky bit for a directory sets a special restriction on deletion of files. Only the owner of the file (and root) can delete files within the directory.

```
[root@node1 ~]# ls -ld /tmp/
```

- **Symbolically: setuid=u+s; setgid=g+s; sticky=o+t**

- **Numerically (fourth preceding digit): setuid=4; setgid=2; sticky=1**

```
[root@node1 ~]# chown g+s dir1
```

```
[root@node1 ~]# chown 2770 dir1
```

Default file permissions:

```
[root@node1 ~]# umask
```

```
0022
```

```
[ali@node1 ~]$ umask
```

```
0002
```

```
[root@node1 ~]# umask 007    (not permanent)
```

```
[root@node1 ~]# vim /etc/bashrc
```

```
[root@node1 ~]# vim /etc/profile
[root@node1 ~]# vim .bashrc
[root@node1 ~]# vim .bash_profile
```

Access control list:

- ACL gives permissions to more than one user or group on a file or a directory.
- ACL sets default permissions for newly created files and directories.
- The file system needs to be mounted with ACL support enabled. XFS file systems have built-in ACL support. Ext4 file systems created on RHEL7 have the acl option enabled by default, but ext4 file systems created in earlier versions of Red Hat Enterprise Linux may need the acl option included with the mount request.

```
[root@node1~]# ls -l file.txt
```

+ (The "+" at the end of the 10-character permission string indicates that there are ACL settings associated with this file)

```
[root@master ~]# getfacl file.txt
```

```
[root@master ~]# getfacl .          (display ACL settings on the working directory)
```

```
[root@master ~]# setfacl -m u:abeer:rw file.txt
```

```
[root@master ~]# setfacl -m o::rw file.txt
```

```
[root@master ~]# setfacl -m g:sales:rw dir1
```

```
[root@master ~]# setfacl -R -m g:sales:rw dir1
```

```
[root@master ~]# setfacl -x u:abeer file.txt
```

```
[root@master ~]# setfacl -b file.txt    (roll back to the default ACL)
```

Managing Networking

Default Network interface names:

- Ethernet interfaces begin with en, WLAN interfaces begin with wl, and WWAN interfaces begin with WW
- The next character(s) represents the type of adapter with an o for on-board, s for hotplug slot, and p for PCI.
- A number N is used to represent an index, ID, or port.
- If the fixed name can not be determined, the traditional names such as ethN will be used.

Example:

eno1, the first embedded network interface

enp2s0, a PCI card network interface

ens226, ethernet hot pluggable slot with id 226

Displaying IP addresses:

```
[root@master ~]# ifconfig
```

```
[root@master ~]# ifconfig eno16777736
```

```
[root@master ~]# ip help
```

```
[root@master ~]# ip addr help
```

```
[root@master ~]# ip addr      (Display IP information)
```

```
[root@master ~]# ip addr show  (Display IP information)
```

```
[root@master ~]# ip addr show eno16777736
```

```
[root@master ~]# ip link show
```

```
[root@master ~]# ip -s link show
```

```
[root@master ~]# ip -s link show eno16777736
```

Configuring IP addresses:

```
[root@client ~]# ifconfig eno16777736 192.168.1.10 netmask 255.255.255.0      (TEMPORARY)
```

-Note: Prefix will be the default if not configured

```
[root@client ~]# ip addr add dev eno16777736 192.168.1.10/24  (Appears as a secondary IP)
```

-Note: Prefix will be /32 if not configured

Configuring Networking with nmcli:

```
[root@client ~]# nmcli
```



```
[root@client ~]# nmcli help
```

```
[root@client ~]# nmcli con show
```

```
[root@client ~]# nmcli con show eno16777736
```

```
[root@client ~]# nmcli connection add con-name testing ifname eno16777736 type ethernet ip4 192.168.1.10/24
```

```
[root@client ~]# nmcli connection down eno16777736
```

```
[root@client ~]# nmcli connection up testing
```

- If the testing connection is lost, the ens224 connection will attempt to autoconnect. To administratively disable an interface and prevent any auto connection, use nmcli dev disconnect DEVICENAME.

```
[root@client ~]# nmcli con mod eno16777736 connection.autoconnect no (Turn off auto connection)
```

```
[root@client ~]# nmcli con mod eno16777736 ipv4.dns 8.8.8.8 (Configure a DNS server)
```

```
[root@client ~]# nmcli con mod eno16777736 +ipv4.dns 4.2.2.3 (Add another DNS server)
```

```
[root@client ~]# nmcli con mod eno16777736 +ipv4.address 10.0.0.1/24 (Add a secondary address)
```

```
[root@client ~]# nmcli con mod eno16777736 ipv4.method manual ipv4.address 192.168.1.10/24 ipv4.gateway 192.168.1.1 ipv4.dns 8.8.8.8
```

- The nmcli con mod will save the setting to the configuration files. To activate the changes, the connection needs to be activated or reactivated.

```
[root@client ~]# nm-connection-editor (GUI version of nmcli)
```

Configuring Networking with nmtui:

```
[root@client ~]# nmtui
```

```
[root@client ~]# nmcli con down eno16777736 ; nmcli con up eno16777736
```

Editing Network Configuration Files:

```
[root@client ~]# vim /etc/sysconfig/network-scripts/ifcfg-eno16777736
```

```
[root@client ~]# nmcli connection reload eno16777736
```

```
[root@client ~]# nmcli connection down eno16777736
```

```
[root@client ~]# nmcli connection up ens224
```

Configuring Host Names:

```
[root@client ~]# hostname
```

```
[root@client ~]# hostname master.redhat.com
```

```
[root@client ~]# vim /etc/hostname
```

```
[root@client ~]# hostnamectl      (Display information about the system)
```

```
[root@client ~]# hostnamectl status  (Display information about the system)
```

```
[root@client ~]# hostnamectl set-hostname www.redhat.com
```

- The static host name is stored in /etc/hostname. Previous versions of RHEL stored the hostname as a variable in the /etc/sysconfig/network file.

Configuring DNS:

```
[root@client ~]# cat /etc/resolv.conf
```

```
[root@client ~]# vim /etc/sysconfig/network-scripts/ifcfg-eno16777736
```

```
DNS1=8.8.8.8
```

```
[root@client ~]# nmcli con mod eno16777736 ipv4.dns 8.8.8.8  (Configure a DNS server)
```

```
[root@client ~]# nmcli con mod eno16777736 +ipv4.dns 4.2.2.3  (Add another DNS server)
```

```
[root@client ~]# vim /etc/hosts
```

Configure Routing:

```
[root@client ~]# route -n      (Display routing table)
```

```
[root@client ~]# ip route      (Display routing table)
```

```
[root@client ~]# ip route show  (Display routing table)
```

```
[root@client ~]# ip route add 20.0.0.0/8 via 192.168.1.1  (Add a static route)
```

```
[root@client ~]# vim /etc/sysconfig/network-scripts/ifcfg-eno16777736
```

```
GATEWAY=192.168.1.1
```

```
[root@client ~]# nmcli con mod eno16777736 ipv4.gateway 192.168.1.1
```

Network tools:

```
[root@client ~]# ping -c 5 127.0.0.1
```

```
[root@client ~]# netstat -i      (Packet information)
```

```
[root@client ~]# netstat -tulpen  (Listening ports)
```

```
[root@client ~]# traceroute 8.8.8.8
```

Storage Management

MBR vs GPT:

- MBR supports a maximum of four primary partitions (max of 63 logical partitions), with a total size of the hard disk of 2Tbyte (2^{31} byte)
- GPT supports a maximum of 128 partitions, with a total size of the hard disk of (2^{72} byte) 8 zebibytes
- MBR is 512 byte .64 byte partition table
.446 byte boot loader
.2 byte magic number

Managing MBR partitions with fdisk:

```
[root@node1 ~]# fdisk -l
```

```
[root@node1 ~]# fdisk /dev/sdb
```

m: for help

l: list known partition types

o: create a new empty DOS partition table

n: add a new partition

p: print the partition table

d: delete a partition

w: write table to disk and exit

- If the newly created partition should have a type other than Linux, enter the t command to change a partition's type.

```
[root@node1 ~]# fdisk -l /dev/sdb
```

```
[root@node1 ~]# partprobe /dev/sdb      (or reload the system)
```

```
[root@node1 ~]# mkfs.ext4 /dev/sdb1
```

```
[root@node1 ~]# mkdir /mydata
```

```
[root@node1 ~]# mount /dev/sdb1 /mydata/
```

```
[root@node1 ~]# df -h
```

```
[root@node1 ~]# df -hT      (show the file system used)
```

```
[root@node1 ~]# mount | grep sdb  (show all mounted systems)
```

- The file system can be mounted using more than one directory.

```
[root@node1 ~]# umount /mydata/    OR [root@node1 ~]# umount /dev/sdb1
```

```
[root@node1 ~]# vim /etc/fstab  
/dev/sdb1    /mydata    ext4  defaults    0 0
```

```
[root@node1 ~]# mount -a    (re-read the fstab file)
```

```
[root@node1 ~]# e2label /dev/sdb1 data
```

```
[root@node1 ~]# e2label /dev/sdb1
```

```
[root@node1 ~]# blkid
```

```
[root@node1 ~]# blkid /dev/sdb1
```

```
[root@node1 ~]# vim /etc/fstab
```

```
LABEL=data    /mydata    ext4  defaults    0 0
```

```
[root@node1 ~]# vim /etc/fstab
```

```
UUID=2cc90e10-8a48-4cbe-8b8b-dd1097ed0ae9 /mydata    ext4  defaults    0 0
```

HINT

```
[root@node1 ~]# dd if=/dev/random of=/dev/sdb bs=1M count=512 (to delete the MBR of the hard disk)
```

- If no type is specified with mkfs command, (ext2) file system will be used.

Managing GPT partitions with gdisk:

```
[root@node1 ~]# gdisk /dev/sdb
```

Swap Space

```
[root@node1 ~]# mkswap /dev/sdb1
```

```
[root@node1 ~]# swapon /dev/sdb1
```

```
[root@node1 ~]# swapon -a    (activate all swap spaces listed in the /etc/fstab file)
```

```
[root@node1 ~]# free -m    (show the swap file system)
```

```
[root@node1 ~]# swapon -s    (show the swap file system)
```

```
[root@node1 ~]# vim /etc/fstab
```

```
/dev/sdb2    swap    ext4  defaults    0 0
```

```
[root@node1 ~]# swapoff /dev/sdb1
```

- A file can be used as a swap file:

```
[root@node1 ~]# dd if=/dev/random of=/swap_file bs=1M count=1024
```

```
[root@node1 ~]# mkswap /swap_file
```

```
[root@node1 ~]# swapon /swap_file
```

- Before moving the swap file to another location, swapoff must be run.

- If data cannot be written to other places, the swapoff will fail, with an error, and the swap space will stay active.

- By default, swap spaces are used in series, meaning that the first activated swap space will be used until it is full, then the kernel will start using the second swap space. Swap space priorities are displayed with `swapon -s`, and can be set with the `pri=` mount option. If swap spaces have the same priority, the kernel will write to them round-robin instead of writing to a single swap space until it is at capacity.

Advanced Storage Solutions

Logical volume management (LVM):

- Volumes can consist of more than one disk.

- Easy resize operation.

- Easy replacement of failing disks.

- Advanced options such as working with snapshots, which allows you to create backups even if they are open.

- Easy to add new volumes.

- Easy to add many volumes.

- Up to 256 logical volume.

SEQUENCE

1- Partition physical storage

2- Create physical volume (PV) (LVM automatically segments PVs into physical extents (PE))

3- Create volume group (VG) (PV can only be allocated to a single VG)

4- Create logical volume (LV)

- Mirroring causes each Logical Extent to map to two Physical Extents.

```
[root@node1 ~]# pvcreate /dev/sdb1 /dev/sdc1 /dev/sdd1 (label the partition for use with LVM)
```

```
[root@node1 ~]# pvdisplay
```

```
[root@node1 ~]# pvdisplay /dev/sdb1
```

```
[root@node1 ~]# pvs
[root@node1 ~]# vgcreate VG1 /dev/sdb /dev/sdc1 /dev/sdd1
[root@node1 ~]# vgdisplay
[root@node1 ~]# vgdisplay VG1
[root@node1 ~]# vgs
[root@node1 ~]# lvcreate -n LV1 -L 2G VG1
[root@node1 ~]# lvdisplay
[root@node1 ~]# lvdisplay /dev/VG1/LV1
[root@node1 ~]# lvs
[root@node1 ~]# mkfs.xfs /dev/VG1/LV1
[root@node1 ~]# mkdir data
[root@node1 ~]# mount /dev/VG1/LV1 data
[root@node1 ~]# df -h
```

Removing a logical volume will destroy any data stored on the logical volume.

```
[root@node1 ~]# lvremove /dev/VG1/LV1 (file system must be unmounted first)
[root@node1 ~]# vgremove VG1
[root@node1 ~]# pvremove /dev/sdb1 /dev/sdc1 /dev/sdd1
```

Extending Logical Volumes (no down time):

```
[root@node1 ~]# pvcreate /dev/sde1
[root@node1 ~]# vgextend VG1 /dev/sde1
[root@node1 ~]# lvextend -L +3G /dev/VG1/LV1
[root@node1 ~]# xfs_growfs /dev/VG1/LV1 (update the file system for XFS file systems)
[root@node1 ~]# resize2fs /dev/VG1/LV1 (update the file system for other file systems)
```

Or:

```
[root@node1 ~]# lvextend -r -L +3G /dev/VG1/LV1 (extend and update in one step)
```

Shrinking a volume group:

- XFS doesn't support shrinking.

```
[root@node1 ~]# umount data
[root@node1 ~]# resize2fs /dev/VG1/LV1 100M
[root@node1 ~]# e2fsck -f /dev/VG1/LV1
[root@node1 ~]# lvreduce --size -3G /dev/VG1/LV1
[root@node1 ~]# lvreduce --size -r -3G /dev/VG1/LV1
[root@node1 ~]# vgreduce VG1 /dev/sde1      (removes sde1 from VG1)
[root@node1 ~]# mount /dev/VG1/LV1 data
```

Device mapper:

- The kernel uses the mapper to connect to storage devices such as LVM, RAID, LUKS.

```
[root@node1 ~]# ll /dev/dm-0
[root@node1 ~]# ll /dev/mapper/VG1-LV1
[root@node1 ~]# ll /dev/VG1/LV1
```

Getting Stratis

```
# yum install stratis-cli stratisd
```

Creating a pool

```
# stratis pool create mypool /dev/vdg
```

Creating filesystems

```
# stratis fs create mypool myfs1
# mkdir myfs1

# mount /dev/stratis/mypool/myfs1 myfs1
```

Snapshots

```
# stratis fs snapshot mypool myfs1 myfs1-experiment
# umount myfs1

# stratis fs destroy mypool myfs1
# stratis fs snapshot mypool myfs1-experiment myfs1

# mount /dev/stratis/mypool/myfs1 myfs1
```

Getting information

```
# stratis pool list
```

To list filesystems within a pool:

```
# stratis fs list mypool
```

To list the blockdevs that make up a pool:

```
# stratis blockdev list mypool
```

These give only minimal information currently, but they will provide more in the future.

Destroying a pool

```
# umount myfs1
```

```
# umount myfs1-experiment (if you created it)
```

```
# stratis fs destroy mypool myfs1
```

```
# stratis fs destroy mypool myfs1-experiment
```

```
# stratis pool destroy mypool
```

VDO Virtual Data Optimizer

```
$ sudo dnf install kmod-kvdo vdo
```

- **vdo** – This is a set of Management tools for Virtual Data Optimizer.
- **kmod-kvdo** – This is a group of Kernel Modules for Virtual Data Optimizer.

After successful installation, start, enable and verify the **vdo** daemon.

```
$ sudo systemctl start vdo
```

```
$ sudo systemctl enable vdo
```

```
$ sudo systemctl status vdo
```

Create a VDO Volume

```
$ sudo vdo create --name=vdo1 --device=/dev/xvdb --vdoLogicalSize=300G
```

LVM Over VDO

```
$ sudo pvcreate /dev/mapper/vdo1
```

```
$ sudo vgcreate vdo1vg /dev/mapper/vdo1
```

```
$ sudo vgdisplay vdo1vg
```

```
$ sudo lvcreate -n vdo1v01 -L 50G vdo1vg
```

```
$ sudo lvcreate -n vdo1v02 -L 50G vdo1vg
```

```
$ sudo mkfs.xfs -K /dev/vdo1vg/vdo1v01
```

```
$ sudo mkfs.xfs -K /dev/vdo1vg/vdo1v02
```